Univerzitet u Sarajevu
Elektrotehnički fakultet
Predmet: Uvod u digitalnu obradu slike
Predmetni profesor: prof. dr Emir Sokić
Autor: Berina Biberović
Akademska godina: 2022/2023.

Tutorijal 8

Deskriptori oblika

Zatvorene konture i klasifikacija na osnovu broja vrhova

Kako bismo mogli klasificirati date elemente na slikama sa laboratorijske vježbe broj 8, potrebno je prvo da izdvojimo konture likova na slici. To možemo uraditi sa funkcijom `cv::findContours`. Trebamo istaći da prvo je potrebno učitati odgovarajuću sliku u *grayscale* obliku, nakon toga uz pomoć funkcije `cv::threshold` koju smo objasnili na tutorijalu 2 i *Otsu*-ove binarizacije prebaciti tu sliku u binarnu sliku. Nad takvom binarnom slikom moguće je da će trebati uraditi istovremeno eroziju i dilataciju kako smo objasnili na prethodnom tutorijalu kako bismo makli šum sa slike.

Nakon toga potrebno je napraviti varijablu tipa `std::vector<std::vector<cv::Point>>` u koju ćemo smjestiti pronađene konture. Pozivamo funkciju `cv::findContours` koja prima sljedeće parametre: prvi parametar je tipa `InputOutputArray` koji predstavlja binarnu sliku; drugi parametar je tipa `OutputArrayOfArrays` koji predstavlja gore navedenu varijablu u koju će biti konture smještene; treći parametar je tipa `int` koji predstavlja princip rada pronalaska kontura; četvrti parametar je tipa `int` i pretstavlja metodu konturne aproksimacije; dok je peti parametar tipa `Point` i on je preklopljen. Dakle, ukoliko je naša binarna slika pod imenom `binarna`, a vektor kontura `konture`, onda možemo pozvati ovu funkciju na sljedeći način:

```
cv::findContours(binarna, konture, cv::RETR_LIST,
    cv::CHAIN_APPROX_SIMPLE);
```

Studentima se predlaže da se igraju sa različitim metodama konturne aproksimacije kako bi vidjeli kakve rezultate dobijaju. Nakon toga, potrebno je aproksimirati dobijene konture sa specificiranom preciznošću, a to možemo uraditi preko funkcije `cv::approxPolyDP` koja aproksimira neku krivu sa drugom krivom koja ima manji broj tjemena. Možemo onda pisati:

```
std::vector< std::vector<cv::Point> > aproksimirane_konture;
aproksimirane_konture.resize(konture.size());

// Pronalazak vrhova kontura:
for(int i = 0; i < konture.size(); i++)
    cv::approxPolyDP(konture.at(i), aproksimirane_konture.at(i), 3,
        true);
```

Nakon toga, u vektoru `aproksimirane_konture` imamo sve konture na slici i potrebno je samo prebrojati koliko vrhova imaju elementi tog vektora i na osnovu toga izvršiti klasifikaciju kontura. Ovakav način klasifikacije nije savršen, i studenti će otkriti koje greške se prave ovom metodom kada budu radili laboratorijsku vježbu 8.

Još neke funkcija koje će možda studentima trebati za zadatak 1 sa laboratorijske vježbe 8 su: `cv::drawContours`, `cv::minEnclosingCircle` i `cv::putText`. Funkcija `cv::drawContours` crta konture na slici tako što joj proslijedimo sliku oblika sa koje smo našli konture, sve konture koje smo našli, indeks konture, boju konture kojom želimo da bude nacrtana i debljinu linije koju želimo. Tako da možemo staviti unutar `for` petlje:

```
cv::drawContours(slika, konture, indeks, cv::Scalar(255, 0, 0), 2);
```

Funkcija `cv::putText` na sličan način radi, prosljeđujemo joj sliku gdje želimo napisati tekst, string koji želimo da se ispiše, tačku na kojoj želimo da se ispiše, font, veličinu i boju teksta. Tako da može ovako glasiti poziv ove funkcije:

```
cv::putText(slika, "Neki tekst", tacka, cv::FONT_HERSHEY_DUPLEX, 0.5,
    cv::Scalar(255, 255, 0));
```

gdje je parametar `tacka` tipa `cv::Point2f`.

Funkcija `cv::minEnclosingCircle` prima tri parametra: niz tačaka (koji kod nas će predstavljati aproksimiranu konturu), tačku `Point2f` koja se prenosi po referenci (izlazni parametar) koja predstavlja sredinu konture i `float` koji se prenosi po referenci (izlazni parametar) koji predstavlja radijus konture.

Cirkularnost i konveksnost geometrijskih oblika

Potrebno je da imamo pronađene aproksimirane konture iz prethodnog dijela tutorijala kako bismo mogli izračunati cirkularnost i konveksnost datih likova na slici. Cirkularnost nekog oblika se računa prema formuli:

$$cirkularnost = 4\pi \frac{površina}{(dužina\ luka)^2}$$

Konveksnost se računa prema formuli:

$$konveksnost = \frac{površina}{površina\ konveksnog\ omotača\ oblika}$$

Za računanje površine možemo koristiti funkciju `cv::contourArea` koja prima gore navedene aproksimirane konture od jednog (!) lika sa slike (obratite pažnju na to šta prosljeđujete u ovu funkciju). Dužina luka se može izračunati preko formule `cv::arcLength` koja prima dva parametra - prvi parametar je isti kao kod funkcije `cv::contourArea`, a drugi parametar je tipa `bool` koji indicira da li je kriva zatvorena ili otvorena - ako je zatvorena onda je ovaj parametar `true`.

Površinu konveksnog omotača nekog oblika možemo izračunati tako što iskoristimo funkciju `cv::convexHull` koja prima dva parametra: aproksimiranu konturu nekog oblika i izlazni parametar tipa `OutputArray`. Onda možemo iskoristiti pronađeni izlazni vektor i na njega primijeniti funkciju `cv::contourArea`.

Računanje momenata konture

U nastavku će biti data funkcija koja računa *Hu* momente likova sa slike:

```
void izracunajMomente(std::vector<std::vector<double>> &hu_vrijednosti,
    std::vector<std::vector<cv::Point>> &konture) {

    // Vektor momenata prvo inicijaliziramo na velicinu konture:
    std::vector<cv::Moments> momenti(konture.size());

    // Racunamo momente preko cv::moments funkcije:
    for(int i = 0; i < konture.size(); i++)
        momenti.at(i) = cv::moments(konture.at(i));

    // Racunanje Hu momenata preko cv::HuMoments funkcije:
    for(int i = 0; i < konture.size(); i++) {
        double hu[7];
        cv::HuMoments(momenti.at(i), hu);
```

```
14          std::vector<double> pomi;
15          for(int j = 0; j < 7; j++) {
16              pomi.push_back(hu[j]);
17          }
18          hu_vrijednosti.push_back(pomi);
19      }
20  }
```

Ova funkcija prima vektor vektora `double`-ova koji su proslijeđeni po referenci i koji će biti upisani na kraju funkcije i predstavljaju *Hu* momente i vektora vektora tačaka koji predstavljaju konture oblika na slici.

Prvo inicijaliziramo vektor intanci klase `cv::Moments` koji je veličine koliko ima likova na slici. Onda računamo momente svake od kontura preko funkcije `cv::moments`. Nakon toga ide glavni dio funkcije u kojem da bismo izračunali *Hu* momente, moramo napraviti unutar `for` petlje prazan niz `double`-ova koji prenosimo kao drugi parametar u funkciji `cv::HuMoments`. Prvi parametar te funkcije su svi do sada izračunati momenti. Nakon toga, uz pomoć pomoćne varijable `pomi` upisujemo u proslijeđeni parametar vrijednosti izračunate u `hu` varijabli. Na kraju je potrebno izvršiti klasifikaciju svih oblika na osnovu pronađenih *Hu* momenata, što se može uraditi na više načina i studenti se podstiču da smisle svoju metodu klasifikacije na osnovu pronađenih *Hu* momenata. Obratiti pažnju na to da je drugi parametar koji prima funkcija `cv::HuMoments` prazan izlazni niz `double`-ova.

Napomene

Korištena tema za prikaz C++ koda na tutorijalima je `solarized`[1].

Literatura

[1] Ethan Schoonover. *Solarized*. `http://ethanschoonover.com/solarized`. Pristup stranici: 20. maj 2023. 2011.